La capa de Transporte

Transporte orientado a conexión: TCP

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios

3.2 Multiplexacción y demultiplexación

3.3 Transporte sin conexión: UDP

3.4 Principios de transferencia de datos fiable

3.5 Transporte orientado a conexión: TCP

- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión

3.7 Mecanismo de control de congestión de TCP

TCP: Transmission Control Protocol

- Especificado en varias RFCs:
  - RFCs 793, 1122, 1323, 2018, y 2581.
- Protocolo de la capa de Transporte.
  - Orientado a conexión, dado todos los servicios que provee.
  - Fiable.
  - Dispone de:
    - Control de flujo
    - Control de congestión.
    - Detección de errores.
    - Retransmisiones.
    - Reconocimientos acumulativos.
    - Números de secuencia.
    - Temporizadores.

TCP: La conexión TCP

- Orientado a conexión.
  - Los procesos de Aplicación, **en sendos sistemas finales**, primero deben establecer-acordar la comunicación.
    - Para ello intercambian segmentos especiales donde establecen los parámetros para la transferencia de datos, iniciando diferentes variables de estado para TCP.
  - Los sistemas intermedios (*routers*, etc.) son **totalmente transparentes** a la "conexión" TCP.
    - No mantienen estados de la conexión TCP.

TCP: La conexión TCP

- Conexión TCP.
  - Full-duplex ➔ Bidireccional simultánea.
    - De flujo de octetos NO estructurados ➔ Flujo de octetos, NO de mensajes.
  - Punto a punto.
    - No permite multidifusión ni difusión.
  - Acuerdo en tres fases.
    - Intercambio de **tres segmentos especiales**.
      - Los dos primeros segmentos (SYN y SYN+ACK): No portan datos de la capa de Aplicación.
      - El tercer segmento (ACK): Puede portar datos de la capa de Aplicación.
    - Finaliza con el establecimiento de las variables de estado y capacidades en *buffers.*

TCP: La conexión TCP

- Conexión TCP establecida.
  - Proceso Cliente pasa datos a la capa TCP.
  - TCP transfiere segmentos que contienen PDUs (datos) de la capa de Aplicación.
    - "Según su propia conveniencia". La especificación TCP es vaga en este sentido.
- *Buffers* TCP ➔ Introducen cierto retardo.

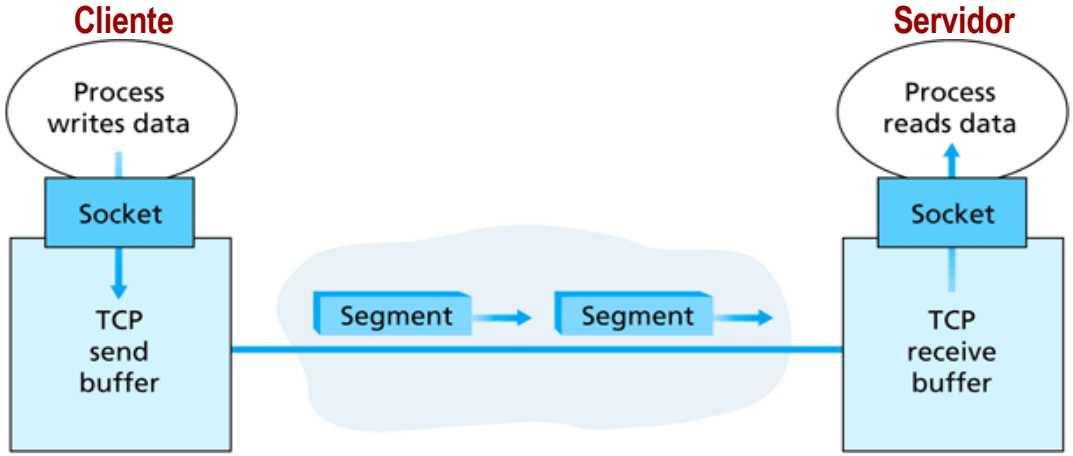


Figure 3.28 ♦ TCP send and receive buffers

Toda conexión TCP requiere en Emisor y Receptor:
- *Buffers*
- Variables
- *Socket* de conexión a un proceso

TCP: MSS: Maximum Segment Size.

- Cantidad máxima de datos que puede portar el segmento TCP.
- Depende del tamaño de la MTU, según:
  - Trama más grande que puede ensamblar el Emisor
  - Trama más grande que puede enviarse entre Emisor y Receptor a través de todos los enlaces entre origen y destino.
    - Hay métodos para determinar la MTU del trayecto y establecer el MSS en correspondencia.
- Garantía de que el segmento TCP se "ajuste" a un único tamaño de trama (PDU_2).
  - Evita tener que segmentar en capas inferiores (fragmentar).
- Se acuerda o se impone en la fase de establecimiento de la conexión.
  - Valor mínimo propuesto por las partes, si no, valor por defecto = 536 octetos.
  - Posibilita que la PDU_{TCP} no necesite un campo de control que indique su longitud.

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios
3.2 Multiplexacción y demultiplexación
3.3 Transporte sin conexión: UDP
3.4 Principios de transferencia de datos fiable
3.5 Transporte orientado a conexión: TCP
- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión
3.7 Mecanismo de control de congestión de TCP

TCP: Estructura del segmento TCP

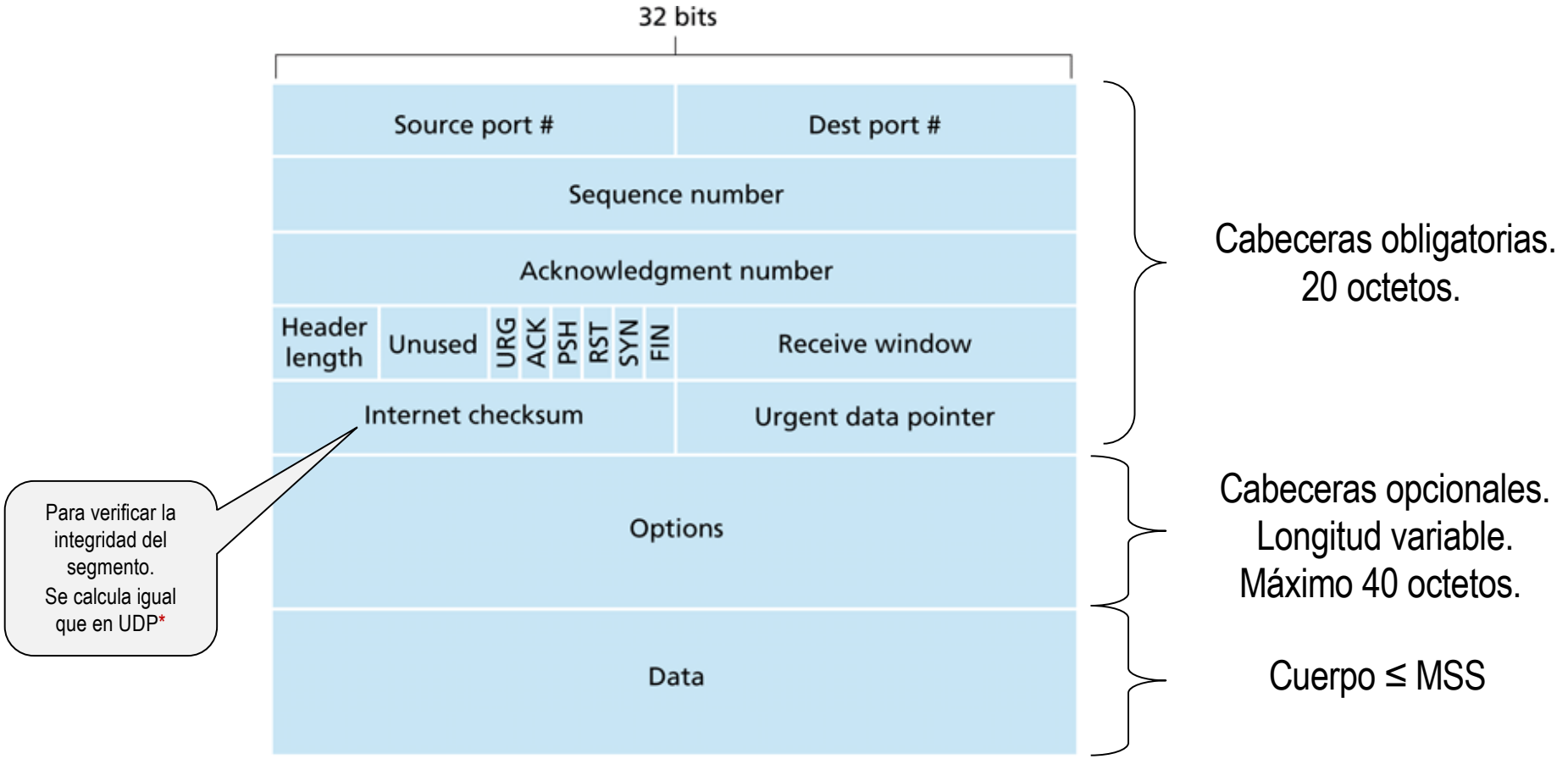


Figure 3.29 ♦ TCP segment structure

TCP: Cabecera del segmento TCP (I)

- Puertos origen y destino (16 bits c/u)
- Nº de secuencia (32 bits)
- Nº de reconocimiento (32 bits)

  ← Para transferencia fiable y control de flujo
- Longitud de cabecera (4 bits)
  - En palabras de 32 bits
  - Si está vacío ➔ 20 octetos (cinco palabras de 32 bits cada una)
- Indicadores (*flags*) (1 bit c/u)
  - URG, ACK, PSH, RST, SYN, FIN
- Ventana de recepción (16 bits) ← Para control de flujo
- Suma de comprobación (16 bits) ← Sobre el segmento completo + pseudo cabecera
- Puntero de datos urgente (16 bits) ← Para indicar ubicación (inicio) de datos urgentes
- Opciones (es opcional, longitud variable) ← Normalmente NO presente*

TCP: Cabecera del segmento TCP (II)

Flags (Indicadores):

- ACK.
  - Indica que el valor del campo “Nº de Reconocimiento” es válido.
- SYN , FIN.
  - Para establecimiento y cierre de conexiones.
- PSH.
  - Indica al Receptor* que pase de inmediato los datos a la capa superior
- URG.
  - Indica que el segmento tiene datos marcados como urgentes por la capa superior
    - El Campo “Puntero de datos urgente” debe contener información válida.

TCP: Estructura del segmento TCP. Cabeceras

Indicadores (*flags*)

- RST.
  - Se utiliza para el reinicio o rechazo de conexión, e indica:
    - Rechazo de solicitud de conexión, cuando:
      - **No existe *socket* activo** para la dirección IP o puerto origen de la solicitud*
      - El servidor NO está "escuchando" por el puerto al que el Cliente envía el segmento SYN.
    - Reinicio de conexión, cuando:
      - Se han producido errores o caída de la conexión.
- Todo *flag* (SYN, ACK, FIN, PSH, URG, RST) con valor lógico:
  - "1" ➔ Activado.
  - "0" ➔ Desactivado

TCP: Cabecera del segmento TCP (III)

- Número de secuencia.
  - Indica lo que se envía.
  - Indica el Nº del primer octeto que porta el segmento respecto al flujo total de octetos.
- Número de reconocimiento.
  - Indica lo que se espera recibir.
  - Indica el Nº de secuencia del siguiente octeto que se espera recibir.
- Ambos SIEMPRE están presentes en TODO segmento TCP.
  - Hacen referencia al flujo de octetos, NO a los segmentos.
    - TCP percibe los datos como un flujo de octetos ordenados pero NO estructurados.
- Elección de los Nº de secuencia.
  - De manera aleatoria por ambas partes de la conexión.
    - Para minimizar posibilidad de coincidencia con segmentos antiguos que estén en la red.

TCP: asentimientos y Nºs de secuencia.

- Confirmaciones acumulativas.
  - TCP sólo confirma (reconoce) hasta el primer octeto que falta en el flujo.
- ¿Qué hace un Host cuando NO recibe los segmentos en orden?
  - Las especificaciones TCP no establecen nada al respecto.
  - Queda a decisión de los desarrolladores.
  - Opciones básicas:
    - Receptor descarta segmentos que no lleguen en secuencia.
      - Se simplifica el diseño del Receptor.
    - Receptor acepta segmentos fuera de secuencia, los almacena en el *buffer* en espera de los faltantes.
      - Solución más eficiente respecto a uso del ancho de banda.
      - Solución normalmente utilizada en la práctica.